SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU EVENT API

WaitEvent, SetEvent, ClearEvent, GetEvent trong mini RTOS

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU EVENT API TRONG MINI RTOS

Từ event bit trong TCB đến task block/unblock qua scheduler

Tài liệu này bám sát đúng code trong repository **Os_Test** sau khi thêm 4 API nền: `WaitEvent()`, `SetEvent()`, `ClearEvent()`, `GetEvent()`. Đây là phần học nền RTOS, chưa đi vào đầy đủ `StatusType` hoặc error semantics của OSEK.

Một câu chốt bản chất của Event: Event là các bit tín hiệu gắn trực tiếp với từng task. Task có thể ngủ bằng `WaitEvent(mask)`; task khác hoặc ISR có thể báo bằng `SetEvent(task, mask)`; khi bit khớp mask đang chờ, task được đưa từ `WAITING` về `READY`.

Mục lục

1. Mục tiêu và bức tranh tổng thể
2. Bản chất của event bitmask
3. Kiến trúc code trong Os_Test
4. Phân tích 4 API
5. Demo task và bảng cấu hình mới
6. Luồng runtime đầy đủ
7. Race safety và critical section
8. Kiểm chứng bằng Renode/GDB
9. Lab step-by-step cho học viên
10. Ứng dụng thực tế trong hệ automotive
11. Debug chuyên sâu bằng GDB
12. Hướng mở rộng lên OSEK/AUTOSAR
13. Câu hỏi kiểm tra hiểu bản chất
14. Điểm dễ nhầm và kết luận

---

1. Mục tiêu và bức tranh tổng thể

Sau bài này, người học cần hiểu được vì sao event là bước tiếp theo tự nhiên sau scheduler. Trước đó task chỉ chạy theo autostart, alarm hoặc schedule table. Với event, task có thể chủ động **ngủ** khi chưa có việc, và chỉ quay lại ready queue khi có tín hiệu phù hợp.

> **Chuỗi runtime cần nắm:**
> `Task Consumer chạy → WaitEvent(mask) → chưa có event → state = WAITING → scheduler chạy Producer → Producer SetEvent(Consumer, mask) → Consumer state = READY → scheduler preempt Producer nếu Consumer ưu tiên cao hơn → Consumer GetEvent/ClearEvent → Consumer xử lý xong`

API	Vai trò	Có block?	Có thể gây schedule?
`WaitEvent(mask)`	Task hiện tại chờ một hoặc nhiều bit event.	Có, nếu chưa có bit khớp.	Có, khi task chuyển sang `WAITING`.
`SetEvent(tid, mask)`	Báo event cho task đích.	Không.	Có, nếu task đích đang chờ và được unblock.
`ClearEvent(mask)`	Xóa các bit event đã xử lý của task hiện tại.	Không.	Không.
`GetEvent(tid, *mask)`	Đọc event_set hiện tại của task đích.	Không.	Không.

1.1 Năng lực đầu ra sau bài học

Bài này không chỉ yêu cầu học viên nhớ tên 4 API. Sau khi học xong, học viên phải đọc được code kernel và tự giải thích chính xác CPU đang chạy ở đâu, task nào còn nằm trong ready queue, task nào đang ngủ, event bit nào đang pending, và vì sao PendSV có hoặc không xảy ra.

Năng lực	Học viên phải làm được	Cách kiểm tra
Đọc data model	Giải thích được `event_set`, `wait_mask`, `state`, `current_priority` trong TCB.	Dừng GDB ở `WaitEvent` / `SetEvent` và in TCB.
Đọc flow runtime	Vẽ lại luồng Consumer block, Producer set event, Consumer được unblock.	So sánh với UART log và sequence diagram.
Hiểu scheduler	Biết vì sao Consumer priority cao hơn Producer thì Consumer có thể preempt Producer sau `SetEvent`.	Breakpoint tại `os_schedule` và quan sát `g_next`.
Hiểu race safety	Chỉ ra race giữa `WaitEvent` và `SetEvent` nếu không có critical section.	Giải thích đoạn `os_irq_save()` / `os_irq_restore()`.
Ứng dụng thiết kế	Biết chọn event cho UART RX, CAN RX, timeout, button, diagnostic request.	Tự đề xuất task/event table cho một module cụ thể.

1.2 Kiến thức cần có trước khi học

- Đã hiểu task state cơ bản: `DORMANT`, `READY`, `RUNNING`, `WAITING`.
- Đã hiểu ready queue theo priority và FIFO trong cùng priority.
- Đã hiểu PendSV là exception thực hiện context switch, còn C scheduler chỉ chọn `g_next`.
- Đã hiểu critical section tối thiểu bằng `PRIMASK`.
- Đã biết đọc log Renode từ USART1.

1.3 Câu hỏi trung tâm của bài

```
Nếu task đang chạy không có việc để làm, nó nên bận chờ bằng vòng lặp polling,
hay nên tự chuyển sang WAITING để CPU chạy task khác?
```

Event API trả lời câu hỏi này. Nó biến một task từ kiểu "luôn chạy để kiểm tra cờ" thành kiểu "ngủ đến khi có tín hiệu". Đây là một bước rất quan trọng trong RTOS vì nó giảm lãng phí CPU, giảm jitter cho task khác, và làm flow phần mềm rõ ràng hơn.

2. Bản chất của event bitmask

2.1 Event không phải queue

Event trong bài này là một **bitmask**. Mỗi bit đại diện cho một điều kiện đã xảy ra. Nếu cùng một bit được set nhiều lần trước khi task clear, trạng thái vẫn chỉ là 1. Vì vậy event phù hợp cho tín hiệu trạng thái như "RX done", "timeout", "button pressed"; không phù hợp nếu cần đếm số lần xảy ra.

```
enum {
    EVENT_RX_DONE = (1u << 0),
    EVENT_TIMEOUT = (1u << 1)
};
```

Giá trị	Ý nghĩa	Diễn giải
`0x00000000`	Không có event pending	Task gọi `WaitEvent()` sẽ block nếu mask cần chờ chưa khớp.
`0x00000001`	`EVENT_RX_DONE`	Consumer có dữ liệu RX cần xử lý.
`0x00000002`	`EVENT_TIMEOUT`	Consumer có timeout cần xử lý.
`0x00000003`	Cả hai bit cùng pending	Task có thể xử lý từng bit rồi clear từng phần.

Event không tự mất. `SetEvent()` chỉ bật bit. Sau khi xử lý, task phải gọi `ClearEvent()`. Nếu không clear, lần `WaitEvent()` tiếp theo sẽ return ngay vì bit vẫn còn pending.

2.2 Event khác polling, semaphore, queue như thế nào?

Cơ chế	Bản chất	Có đếm số lần?	Dùng khi nào?	Ví dụ
Polling flag	Task tự đọc cờ liên tục.	Không.	Demo rất nhỏ, hoặc vòng lặp idle đơn giản.	`while (!rx_done) {}`
Event bit	Bit pending gắn với một task.	Không, mỗi bit chỉ 0/1.	Báo "điều kiện đã xảy ra".	RX done, timeout, wakeup, new diagnostic request.
Semaphore	Token/counter đồng bộ tài nguyên hoặc số lần xảy ra.	Có, nếu là counting semaphore.	Đếm số buffer, số packet, số resource.	3 buffer RX đang chờ xử lý.
Queue	Danh sách message/data có thứ tự.	Có, từng phần tử.	Truyền dữ liệu, không chỉ báo trạng thái.	Queue CAN frame hoặc UART byte stream.

Cách chọn đúng: Nếu chỉ cần đánh thức task vì "có việc mới", dùng event. Nếu cần truyền payload, event chỉ nên đóng vai trò wakeup; payload nên nằm trong ring buffer hoặc queue riêng. Nếu cần đếm từng lần xảy ra, không dùng event bit đơn thuần.

2.3 Auto-clear hay manual-clear?

Bản hiện tại dùng **manual-clear**: `WaitEvent()` không xóa bit, task phải tự gọi `ClearEvent()` sau khi xử lý. Đây là cách giúp task kiểm tra được nhiều bit cùng lúc và quyết định clear bit nào.

Kiểu	Cách hoạt động	Ưu điểm	Rủi ro
Manual-clear	`SetEvent` bật bit, task xử lý xong gọi `ClearEvent`.	Rõ ràng, task kiểm soát thời điểm xóa.	Quên clear thì `WaitEvent` sau return ngay.
Auto-clear	Kernel tự xóa bit khi `WaitEvent` return.	Code task ngắn hơn.	Dễ mất thông tin nếu task cần đọc nhiều bit hoặc xử lý theo thứ tự riêng.

2.4 Công thức bitmask cần nhớ

```
/* Set một hoặc nhiều bit */
event_set |= mask;

/* Kiểm tra có ít nhất một bit khớp */
if ((event_set & wait_mask) != 0u) { ... }

/* Clear một hoặc nhiều bit */
event_set &= ~mask;

/* Chờ một trong hai event */
WaitEvent(EVENT_RX_DONE | EVENT_TIMEOUT);
```

3. Kiến trúc code trong Os_Test

3.1 Thay đổi trong TCB

Event được gắn vào từng task qua hai field mới trong `TCB_t` tại `OS/inc/os_kernel.h`:

```
typedef struct TCB {
    uint32_t *sp;                /* field đầu tiên cho PendSV/SVC */
    ...
    volatile uint8_t state;
    volatile EventMaskType event_set; /* Các event bit đã được báo */
    volatile EventMaskType wait_mask; /* Các event bit task đang chờ */
} TCB_t;
```

Field	Ai ghi?	Ai đọc?	Ý nghĩa runtime
`event_set`	`SetEvent()`, `ClearEvent()`	`WaitEvent()`, `GetEvent()`	Các bit event đang pending của task.
`wait_mask`	`WaitEvent()`, `SetEvent()`	`SetEvent()`	Mask mà task đang block chờ.
`state`	Scheduler và event API	Scheduler	Thêm luồng thực tế cho `OS_WAITING`.

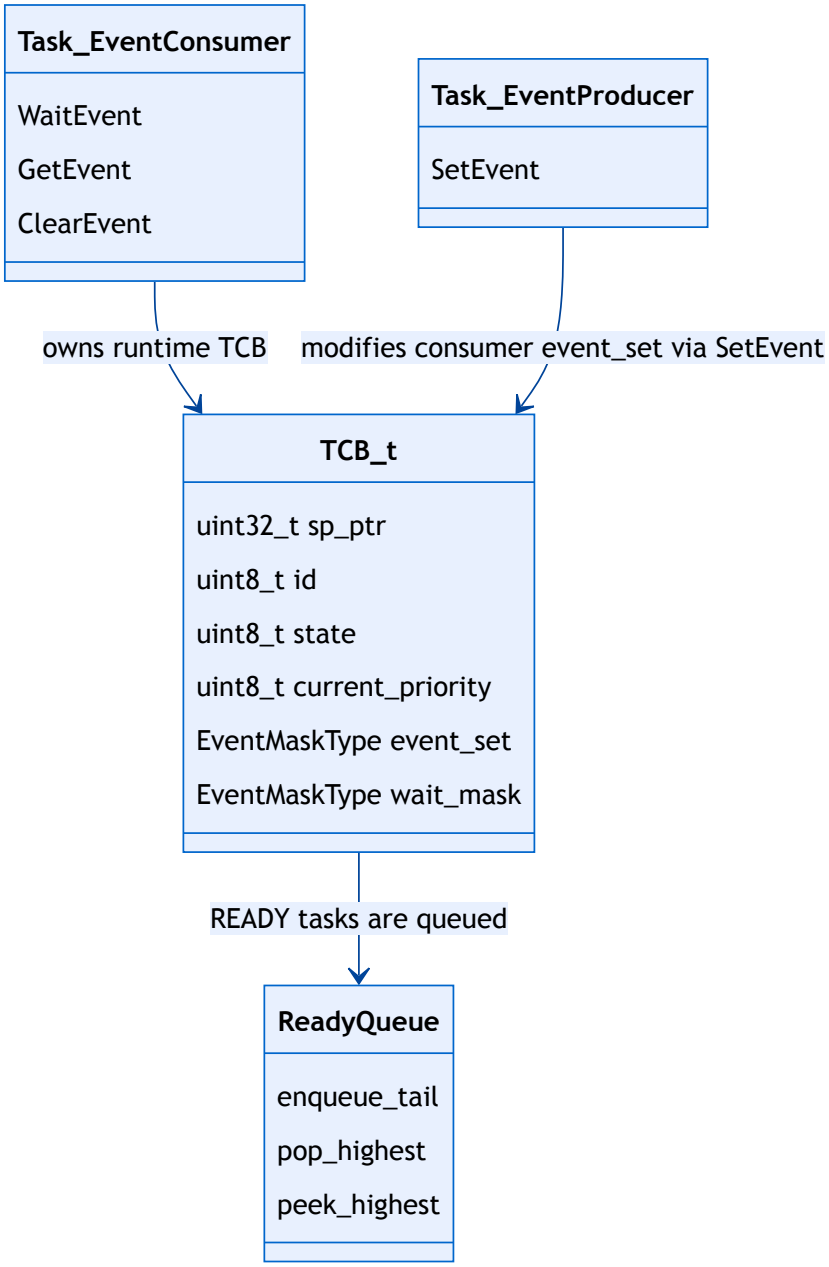
3.2 Mapping file

File	Vai trò trong bài event
`OS/inc/os_types.h`	Định nghĩa `EventMaskType`.
`OS/inc/os_kernel.h`	Thêm field event vào TCB và prototype 4 API.
`OS/src/os_kernel.c`	Hiện thực `WaitEvent`, `SetEvent`, `ClearEvent`, `GetEvent`.
`Config/os_app_cfg.h`	Định nghĩa task ID và event mask demo.
`Config/os_app_cfg.c`	Tạo lại bảng task tối giản: Boot, Consumer, Producer, Idle.
`app/App_Task.c`	Demo dùng đủ 4 API và in UART log để test.
`test_os.py`	Build, chạy Renode, validate log, GDB smoke.

3.3 Data ownership: ai sở hữu event?

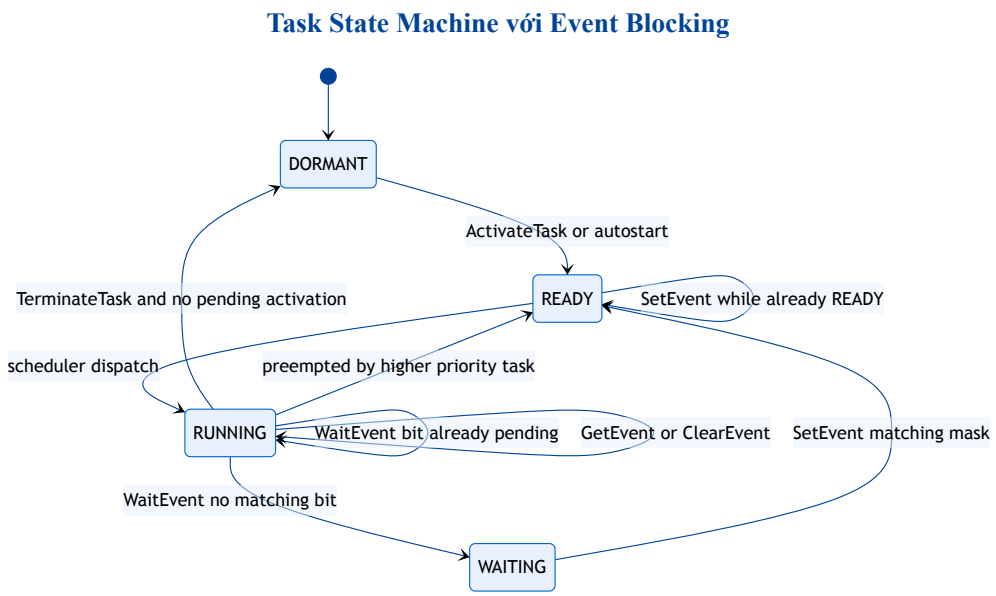
Trong thiết kế này, event không phải object độc lập. Event thuộc về task nhận. Vì vậy `Task_EventConsumer` sở hữu `event_set` của nó, còn `Task_EventProducer` chỉ có quyền bật bit thông qua `SetEvent(TASK_EVENT_CONSUMER, mask)`.

Class/Data Model: TCB và Event Fields



Event bits sống trong TCB của task nhận, không sống trong Producer.

3.4 State machine của task khi có event



Điểm mới của bài này là cạnh RUNNING → WAITING và WAITING → READY.

3.5 Invariant của kernel sau khi thêm event

Invariant	Lý do cần giữ	Nếu vi phạm sẽ ra sao?
Task `WAITING` không nằm trong ready queue.	Scheduler không được chọn task đang ngủ.	Task bị chạy dù event chưa đến.
Khi task được unblock, phải enqueue đúng priority hiện tại.	Scheduler vẫn giữ luật priority.	Task bị đói CPU hoặc chạy sai thứ tự.
`event_set` chỉ bị xóa bởi task hiện tại qua `ClearEvent`.	Task nhận kiểm soát lifecycle của event.	Producer/ISR có thể làm mất event trước khi task xử lý.
`wait_mask` được reset khi task được unblock.	Task không còn đang ngủ vì mask cũ.	Debug state bị sai, unblock sau có thể bị hiểu nhầm.
Không gọi scheduler khi đang trong ISR nesting.	Context switch thật phải defer qua PendSV/exit ISR.	Đổi context ở thời điểm không an toàn.

4. Phân tích 4 API

4.1 WaitEvent(mask)

`WaitEvent()` chỉ tác động lên task hiện tại. Nếu event đã có sẵn, hàm return ngay. Nếu chưa có, task bị chuyển sang `OS_WAITING` và scheduler chọn task khác.

```
void WaitEvent(EventMaskType mask)
{
    if ((current->event_set & mask) != 0u) {
        return;
    }

    current->wait_mask = mask;
    current->state = OS_WAITING;
    os_schedule();
}
```

> **Điểm quan trọng:** task đang `WAITING` không nằm trong ready queue. Nó chỉ quay lại ready queue khi `SetEvent()` phát một bit khớp với `wait_mask`.

4.2 SetEvent(tid, mask)

`SetEvent()` OR bit vào task đích. Nếu task đích đang chờ và có bit khớp, task được chuyển về `READY`, enqueue theo priority, rồi scheduler có thể dispatch.

```
task->event_set |= mask;

if ((task->state == OS_WAITING) &&
    ((task->event_set & task->wait_mask) != 0u)) {
    task->wait_mask = 0u;
    task->state = OS_READY;
    os_ready_add(task->id, task->current_priority);
    os_schedule();
}
```

4.3 GetEvent(tid, *mask)

`GetEvent()` chỉ copy `event_set` ra biến của caller. Nó không clear, không block, không gọi scheduler.

```
EventMaskType events;
GetEvent(TASK_EVENT_CONSUMER, &events);
```

4.4 ClearEvent(mask)

`ClearEvent()` xóa bit event của task hiện tại bằng phép AND với bit đảo.

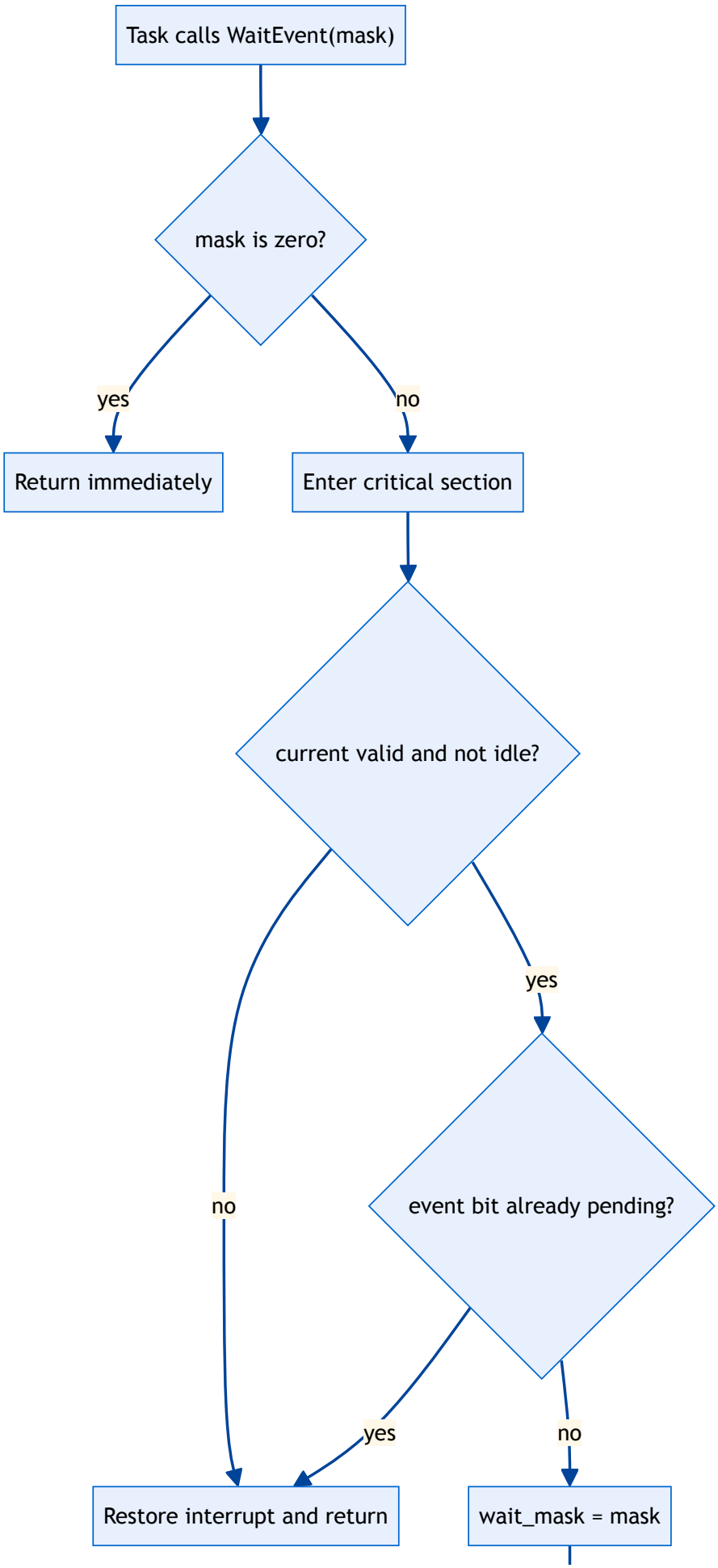
```
current->event_set &= ~mask;
```

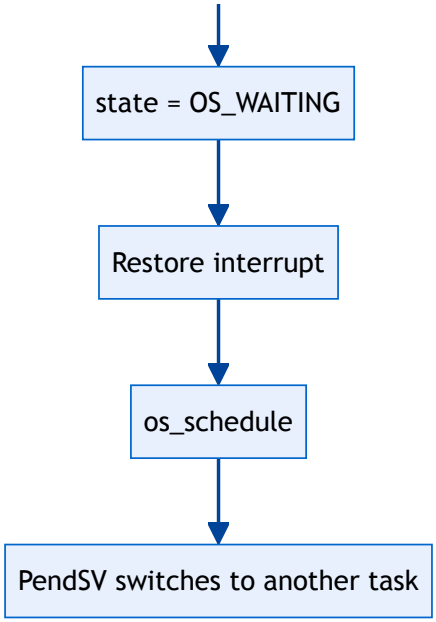
Trong bản nền này, API đang dùng kiểu `void` để đồng bộ với style kernel hiện tại. Các rule lỗi kiểu gọi từ ISR, task ID sai, access right, extended/basic task sẽ để sang cụm OSEK API sau.

4.5 Thuật toán đầy đủ của WaitEvent theo từng bước

Bước	Code/logic	Ý nghĩa chuyên sâu
1	`if (mask == 0u) return;`	Mask rỗng không có điều kiện chờ. Bản nền bỏ qua thay vì báo lỗi.
2	`primask = os_irq_save();`	Bảo vệ đoạn check event và chuyển state. Đây là vùng chống lost wakeup.
3	`current = g_current;`	`WaitEvent` luôn áp dụng cho task đang chạy, không nhận task ID.
4	`if ((event_set & mask) != 0u) return;`	Nếu event đã đến trước khi task chờ, task không được block.
5	`wait_mask = mask;`	Ghi lại điều kiện đánh thức task.
6	`state = OS_WAITING;`	Task rời khỏi nhóm task runnable.
7	`os_irq_restore(primask); os_schedule();`	Mở interrupt đúng trạng thái cũ rồi gọi scheduler để chọn task khác.

Activity: WaitEvent()

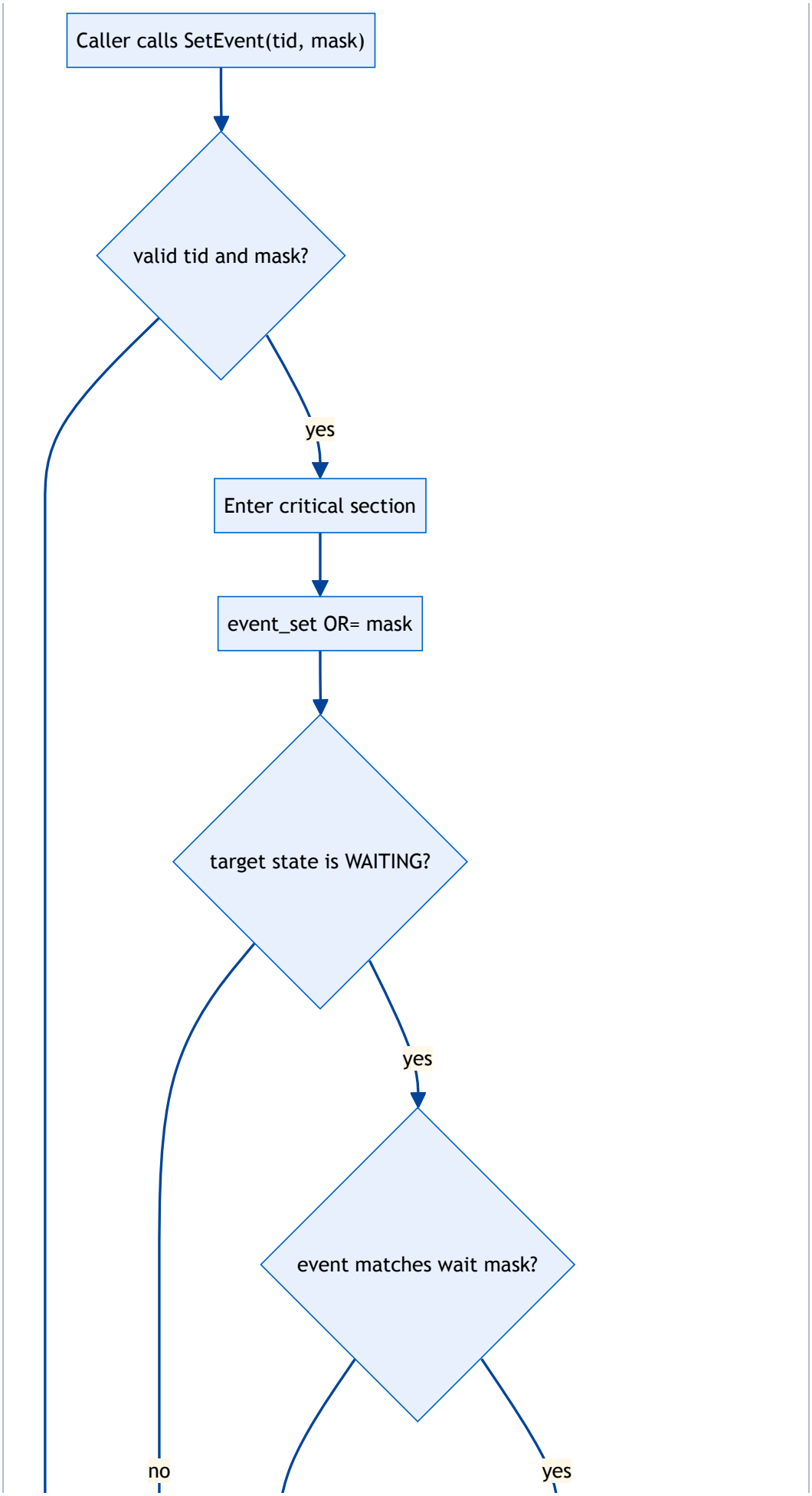

Task calls WaitEvent(mask)
mask is zero?
yes
no
Return immediately
Enter critical section
current valid and not idle?
yes
no
event bit already pending?
yes
no
Restore interrupt and return
wait_mask = mask

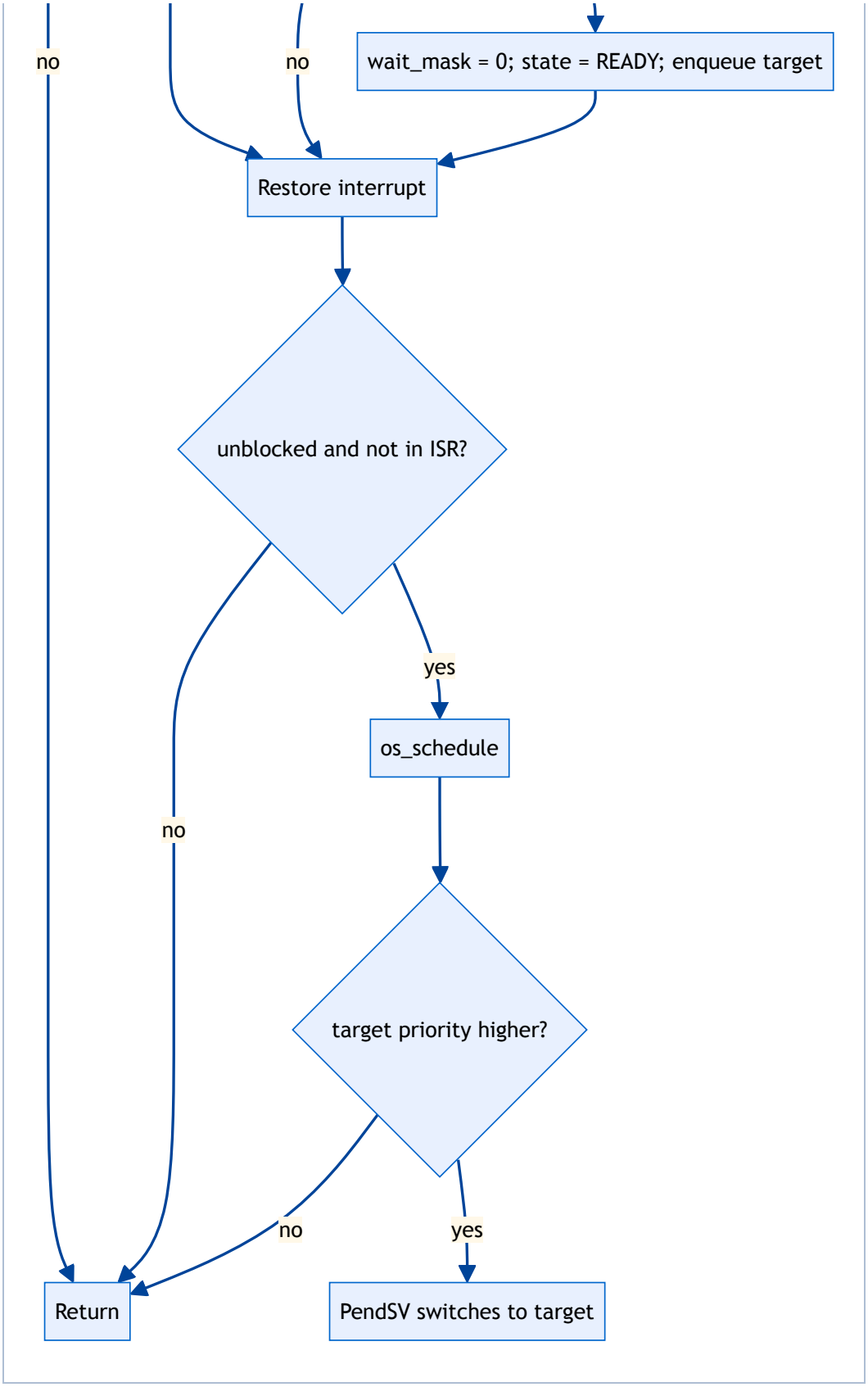


4.6 Thuật toán đầy đủ của SetEvent theo từng bước

Bước	Code/logic	Ý nghĩa chuyên sâu
1	Kiểm tra `mask`, `tid`, idle task.	Bỏ qua input vô nghĩa trong bản nền.
2	Vào critical section.	Bảo vệ TCB và ready queue.
3	`task->event_set |= mask;`	Event là sticky bit: set rồi giữ nguyên cho đến khi clear.
4	Kiểm tra `state == OS_WAITING`.	Chỉ task đang ngủ mới cần unblock. Task READY/RUNNING chỉ nhận bit pending.
5	Kiểm tra `event_set & wait_mask`.	Chỉ unblock nếu bit mới hoặc bit đã pending khớp điều kiện chờ.
6	`state = OS_READY; os_ready_add(...)`.	Đưa task trở lại thế giới runnable.
7	Nếu không ở ISR, gọi scheduler.	Task được đánh thức có thể priority cao hơn current.

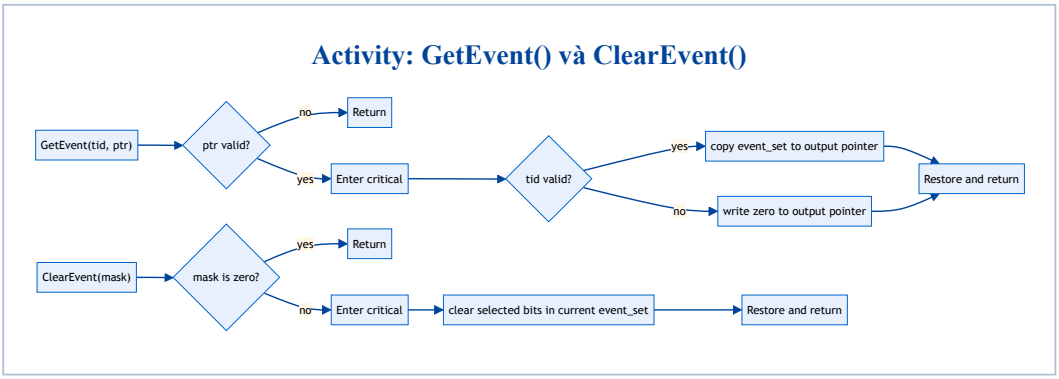
Activity: SetEvent()


Caller calls SetEvent(tid, mask)
valid tid and mask?
yes
Enter critical section
event_set OR= mask
target state is WAITING?
yes
event matches wait mask?
no
yes


no
no
wait_mask = 0; state = READY; enqueue target
Restore interrupt
unblocked and not in ISR?
yes
os_schedule
no
target priority higher?
no
yes
Return
PendSV switches to target

4.7 Thuật toán GetEvent và ClearEvent

API	Luồng chi tiết	Điểm cần dạy kỹ
`GetEvent(tid, &mask)`	Vào critical section, nếu `tid` hợp lệ thì copy `tcb[tid].event_set`, nếu sai thì trả `0`.	Đọc không làm thay đổi trạng thái. Học viên hay nhầm GetEvent giống consume event.
`ClearEvent(mask)`	Vào critical section, lấy `g_current`, clear bit bằng `event_set &= ~mask`.	Clear chỉ áp dụng cho task hiện tại trong bản nền. Clear không gọi scheduler.



5. Demo task và bảng cấu hình mới

5.1 Task set

Task	Priority	Autostart	Vai trò
`Task_EventBoot`	`OS_PRIO_CRITICAL`	Có	Khởi tạo UART log và in profile.
`Task_EventConsumer`	`OS_PRIO_HIGH`	Có	Chờ `EVENT_RX_DONE`, xử lý, clear, rồi chờ `EVENT_TIMEOUT`.
`Task_EventProducer`	`OS_PRIO_MEDIUM`	Có	Set event cho Consumer hai lần.
`Task_Idle`	`OS_PRIO_IDLE`	Có	Fallback khi không còn task ready.

5.2 Vì sao Consumer ưu tiên cao hơn Producer?

Đây là cách demo rõ nhất cho unblock + preemption. Consumer chạy trước, gọi `WaitEvent()` rồi block. Producer bắt đầu chạy và gọi `SetEvent()`. Vì Consumer có priority cao hơn Producer, scheduler có quyền ngắt Producer để Consumer xử lý event ngay.

5.3 Step-by-step đọc bảng config

Học viên nên đọc cấu hình theo đúng thứ tự sau, vì đây là cách kernel cũng "nhìn" hệ thống lúc `OS_Init()`.

Bước	File/dòng cần đọc	Câu hỏi cần trả lời
1	`Config/os_app_cfg.h`	Có những task ID nào? Event mask nào được định nghĩa?
2	`Config/os_app_cfg.c` - `Task_EventBoot`	Task nào chạy đầu tiên và vì sao? Priority của nó là gì?
3	`Config/os_app_cfg.c` - `Task_EventConsumer`	Vì sao Consumer có priority cao hơn Producer?
4	`Config/os_app_cfg.c` - `Task_EventProducer`	Producer có autostart không? Khi nào nó được chạy?
5	`Config/os_app_cfg.c` - `Task_Idle`	Idle có vai trò gì nếu tất cả task còn lại đã terminate hoặc waiting?
6	`OS/src/os_kernel.c` - `OS_Init()`	Task autostart được đưa vào ready queue như thế nào?

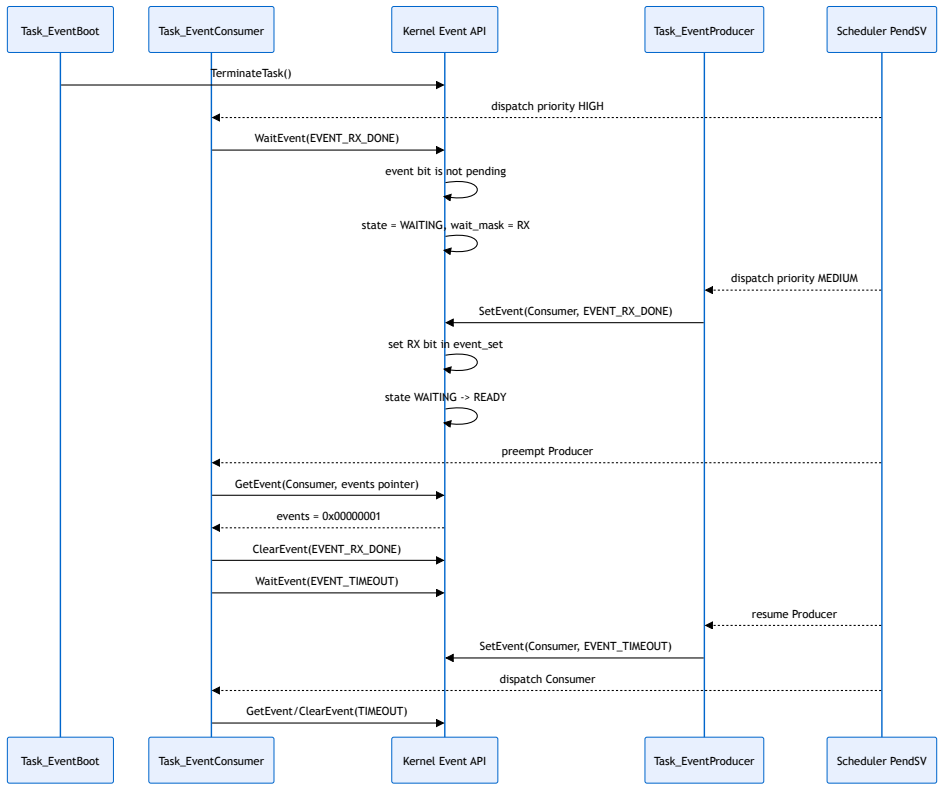
5.4 Step-by-step đọc code task demo

Thứ tự runtime	Task	Code chính	Điều xảy ra trong kernel
1	Boot	`Evcu_LogInit();` `TerminateTask();`	Boot kết thúc, scheduler chọn task READY priority cao nhất tiếp theo.
2	Consumer	`WaitEvent(EVENT_RX_DONE);`	Chưa có bit RX, Consumer chuyển `RUNNING → WAITING` .
3	Producer	`SetEvent(CONSUMER, EVENT_RX_DONE);`	Consumer chuyển `WAITING → READY` ; vì priority cao hơn, Producer bị preempt.
4	Consumer	`GetEvent(...);` `ClearEvent(RX);`	Đọc được `0x1` , rồi clear về `0x0` .
5	Consumer	`WaitEvent(EVENT_TIMEOUT);`	Consumer ngủ lần hai.
6	Producer	`SetEvent(CONSUMER, EVENT_TIMEOUT);`	Consumer được đánh thức lần hai.
7	Consumer	`GetEvent(...);` `ClearEvent(TIMEOUT);`	Đọc được `0x2` , rồi clear về `0x0` .
8	Consumer/Producer	`TerminateTask();`	Các task demo kết thúc, Idle giữ CPU.

5.5 Timeline trạng thái TCB trong demo

Mốc	Consumer state	Consumer event_set	Consumer wait_mask	Task đang chạy
Sau OS_Init	READY	`0x0`	`0x0`	Boot
Consumer gọi WaitEvent RX	WAITING	`0x0`	`0x1`	Producer sau khi schedule
Producer SetEvent RX	READY rồi RUNNING	`0x1`	`0x0`	Consumer sau preempt
Consumer ClearEvent RX	RUNNING	`0x0`	`0x0`	Consumer
Consumer gọi WaitEvent TIMEOUT	WAITING	`0x0`	`0x2`	Producer
Producer SetEvent TIMEOUT	READY rồi RUNNING	`0x2`	`0x0`	Consumer
Consumer ClearEvent TIMEOUT	RUNNING	`0x0`	`0x0`	Consumer

6. Luồng runtime đầy đủ

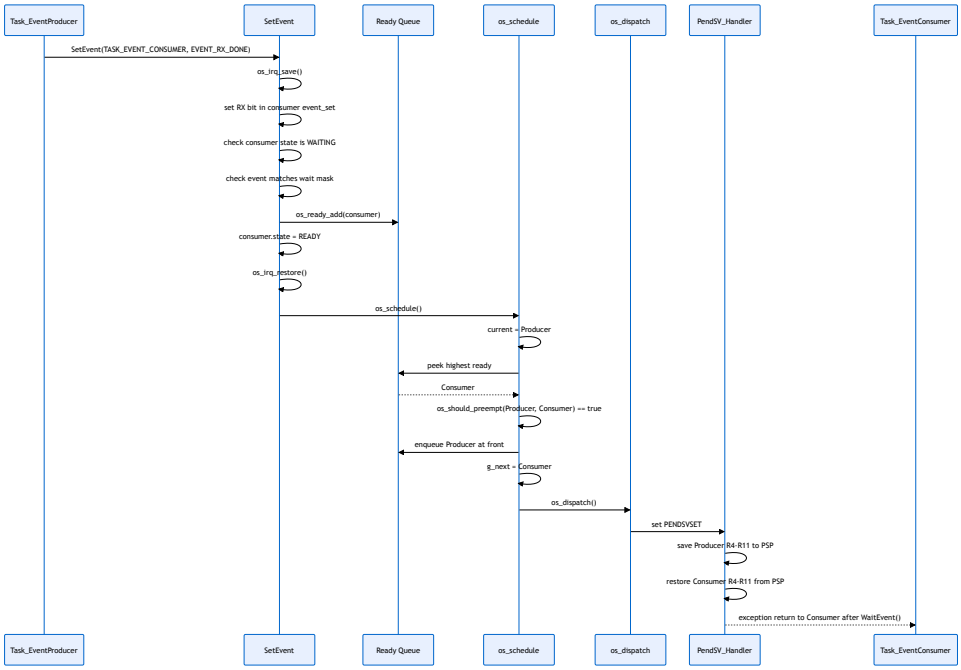


Producer không cần biết Consumer đang chạy hay ngủ. Nó chỉ set bit; kernel quyết định unblock và schedule.

6.1 Sequence đầy đủ có scheduler, g_current/g_next và PendSV

Sequence dưới đây chi tiết hơn: nó cho thấy API event không tự copy stack hay tự nhảy vào task khác. Event API chỉ đổi state và enqueue. Scheduler chọn `g_next`, sau đó `os_dispatch()` request PendSV. Context switch thật vẫn nằm trong assembly `PendSV_Handler`.

Full Sequence: từ SetEvent đến PendSV context switch



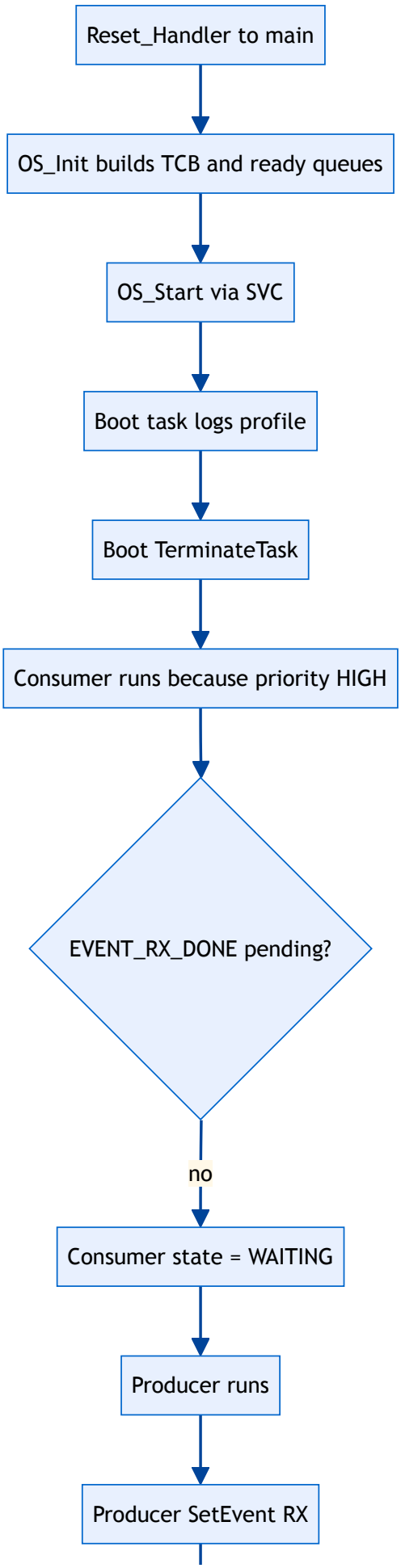
Điểm then chốt: Consumer chạy tiếp ngay sau dòng `WaitEvent(EVENT_RX_DONE)`, không chạy lại từ đầu task.

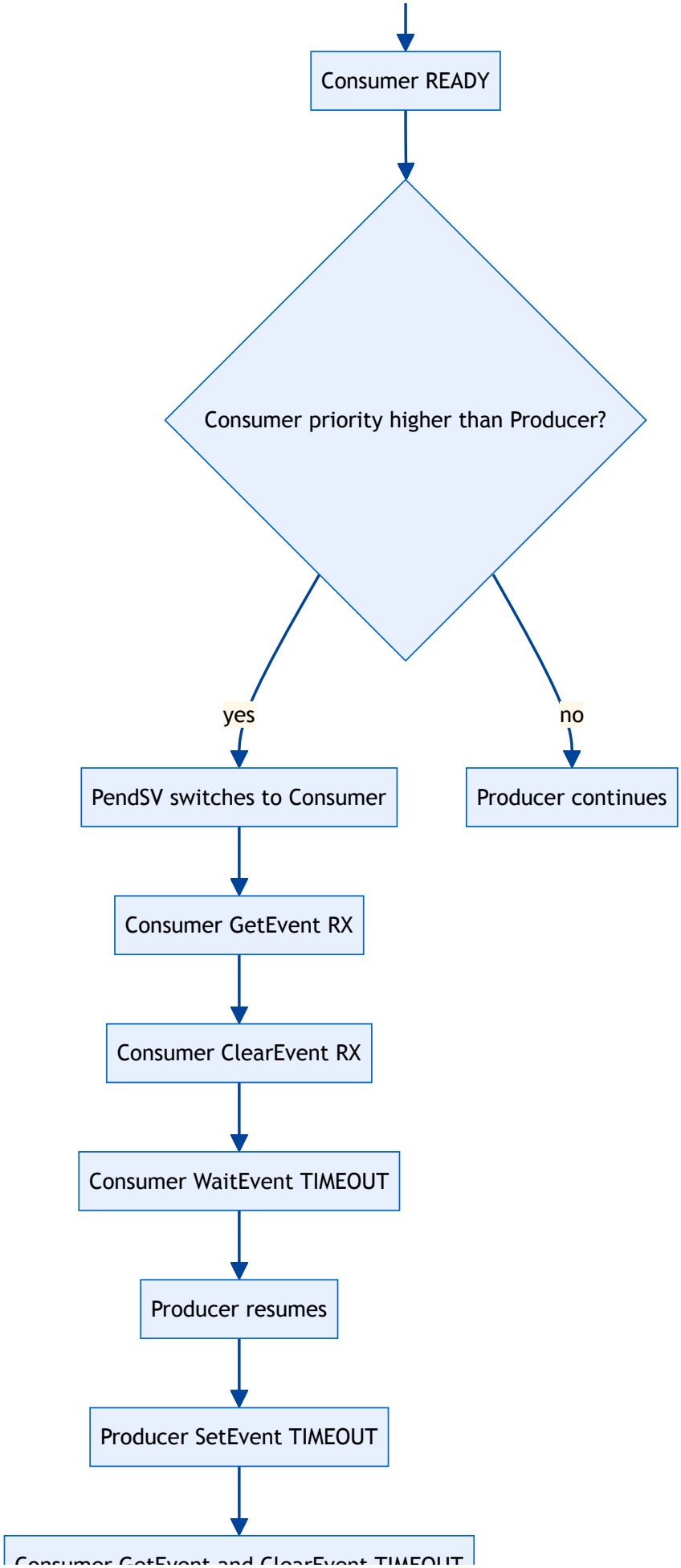
6.2 Timeline log kỳ vọng

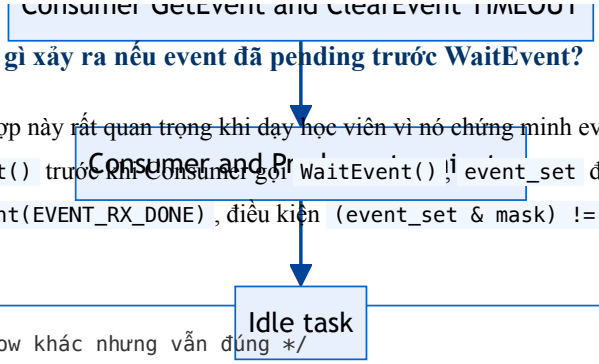
Thứ tự	Log	Ý nghĩa
1	`[CONSUMER] WaitEvent(EVENT_RX_DONE)`	Consumer chưa có RX event nên block.
2	`[PRODUCER] SetEvent(EVENT_RX_DONE)`	Producer phát bit RX cho Consumer.
3	`[CONSUMER] GetEvent after RX mask=0x00000001`	Consumer được đánh thức và đọc đúng bit RX.
4	`[CONSUMER] GetEvent after ClearEvent(RX) mask=0x00000000`	RX bit đã được clear.
5	`[CONSUMER] WaitEvent(EVENT_TIMEOUT)`	Consumer ngủ lần hai.
6	`[PRODUCER] resumed after consumer blocked again`	Producer tiếp tục sau khi Consumer block.
7	`[PRODUCER] SetEvent(EVENT_TIMEOUT)`	Producer phát bit timeout.
8	`[CONSUMER] GetEvent after TIMEOUT mask=0x00000002`	Consumer đọc đúng bit timeout.
9	`[CONSUMER] GetEvent after ClearEvent(TIMEOUT) mask=0x00000000`	Timeout bit đã được clear.

6.3 Activity toàn hệ thống

Activity: Demo Event API End-to-End


Reset_Handler to main
OS_Init builds TCB and ready queues
OS_Start via SVC
Boot task logs profile
Boot TerminateTask
Consumer runs because priority HIGH
EVENT_RX_DONE pending?
no
Consumer state = WAITING
Producer runs
Producer SetEvent RX


Consumer READY
Consumer priority higher than Producer?
yes
no
PendSV switches to Consumer
Producer continues
Consumer GetEvent RX
Consumer ClearEvent RX
Consumer WaitEvent TIMEOUT
Producer resumes
Producer SetEvent TIMEOUT



6.4 Điều gì xảy ra nếu event đã pending trước WaitEvent?

Trường hợp này rất quan trọng khi dạy học viên vì nó chứng minh event là sticky bit. Nếu Producer gọi `SetEvent()` trước khi Consumer gọi `WaitEvent()`, `event_set` đã có bit. Khi Consumer gọi `WaitEvent(EVENT_RX_DONE)`, điều kiện `(event_set & mask) != 0` đúng, nên Consumer không block.

```
/* Flow khác nhưng vẫn đúng */
SetEvent(TASK_EVENT_CONSUMER, EVENT_RX_DONE);

/* Sau đó Consumer chạy */
WaitEvent(EVENT_RX_DONE);  /* return ngay */
GetEvent(TASK_EVENT_CONSUMER, &events); /* events có bit RX */
```

> Đây là cơ chế tránh "lost wakeup" ở mức semantic. Event đến trước vẫn được lưu lại trong `event_set` cho đến khi task nhận clear.

7. Race safety và critical section

Cả bốn API đều chạm vào state dùng chung của kernel: `event_set`, `wait_mask`, `state`, ready queue. Vì các field này có thể bị truy cập từ task context và sau này từ ISR, mỗi API đều dùng critical section nội bộ.

```
uint32_t primask = os_irq_save();
/* đọc/sửa TCB và ready queue */
os_irq_restore(primask);
```

Race nếu không khóa	Hậu quả	Cách bản hiện tại tránh
`WaitEvent()` kiểm tra chưa có event, đúng lúc ISR/task khác `SetEvent()`.	Task có thể ngủ dù event đã đến.	Kiểm tra `event_set` và chuyển `WAITING` trong cùng critical section.
`SetEvent()` unblock task trong lúc scheduler đang đọc ready queue.	Ready queue sai count/head/tail.	Enqueue trong critical section.
`ClearEvent()` và `SetEvent()` cùng sửa một bitmask.	Mất event hoặc clear nhầm event mới.	Bitmask update được bảo vệ bởi `os_irq_save()`.

8. Kiểm chứng bằng Renode/GDB

8.1 Chạy test tự động

```
cd /Users/phamvanvu/Projects/AUTOSAR/OSEK-RTOS/Os_Test
python3 test_os.py
```

Kết quả pass cần có hai phần:

```
PASS: Event API OK: Consumer block bang WaitEvent, Producer SetEvent danh thuc,
GetEvent doc dung mask 0x1/0x2, ClearEvent xoa ve 0.

PASS: GDB attached vao Renode port ..., cai hardware breakpoint
Task_EventConsumer/SetEvent va doc symbol g_current/g_next
```

8.2 Ý nghĩa GDB smoke

GDB smoke không chỉ kiểm tra Renode chạy được. Nó chứng minh ELF còn đủ debug symbol và debugger có thể đặt hardware breakpoint vào `Task_EventConsumer` và `SetEvent`. Đây là hai điểm tốt nhất để debug bài event: một điểm ở task chờ, một điểm ở API đánh thức.

9. Lab step-by-step cho học viên

9.1 Lab 1 - Chạy và đọc log

Mục tiêu của lab này là học viên nhìn thấy thứ tự execution thực tế, không chỉ đọc code tĩnh.

Bước	Lệnh/thao tác	Kết quả cần thấy	Ý nghĩa
1	`cd /Users/phamvanvu/Projects/AUTOSAR/OSEK-RTOS/Os_Test`	Đứng đúng thư mục project.	Test script dùng path tương đối trong project.
2	`make clean && make -j4 all`	Sinh `build/os_test/os_test.elf`.	Xác nhận code event build được.
3	`python3 test_os.py`	PASS Event API và PASS GDB smoke.	Xác nhận flow runtime đúng trên Renode.
4	Mở log Renode trong `build/os_test/renode_mixed_schedule_table.log` nếu script đang dùng tên cũ.	Các dòng `[CONSUMER]`, `[PRODUCER]`.	Đọc thứ tự thực tế của task.

9.2 Lab 2 - Đổi priority để quan sát khác biệt

Trong `Config/os_app_cfg.c`, thử đổi `Task_EventConsumer` từ `OS_PRIO_HIGH` xuống bằng hoặc thấp hơn Producer. Học viên cần dự đoán điều gì xảy ra trước khi chạy.

Thí nghiệm	Dự đoán	Giải thích
Consumer HIGH, Producer MEDIUM	Consumer preempt Producer ngay sau `SetEvent`.	Task được unblock có priority cao hơn current.
Consumer MEDIUM, Producer MEDIUM	Producer có thể tiếp tục đến điểm schedule tiếp theo tùy policy và FIFO.	Cùng priority không được xem là higher-priority preemption.
Consumer LOW, Producer MEDIUM	Producer tiếp tục chạy, Consumer chờ đến khi Producer terminate/block.	Unblock không đồng nghĩa chạy ngay nếu priority thấp hơn.

> Khi làm thí nghiệm này, nhớ khôi phục priority demo ban đầu để test tự động quay lại pass.

9.3 Lab 3 - Quên ClearEvent để thấy lỗi logic

Thử comment dòng `ClearEvent(EVENT_RX_DONE)`. Lúc đó `event_set` vẫn giữ bit RX. Nếu Consumer sau này lại `WaitEvent(EVENT_RX_DONE)`, nó sẽ không ngủ nữa mà return ngay. Đây là lỗi rất thường gặp trong event kiểu manual-clear.

```
/* Sai có chủ đích để học */
GetEvent(TASK_EVENT_CONSUMER, &events);
/* ClearEvent(EVENT_RX_DONE); bị bỏ quên */
```

9.4 Lab 4 - Set hai event cùng lúc

Thay Producer thành:

```
SetEvent(TASK_EVENT_CONSUMER, EVENT_RX_DONE | EVENT_TIMEOUT);
```

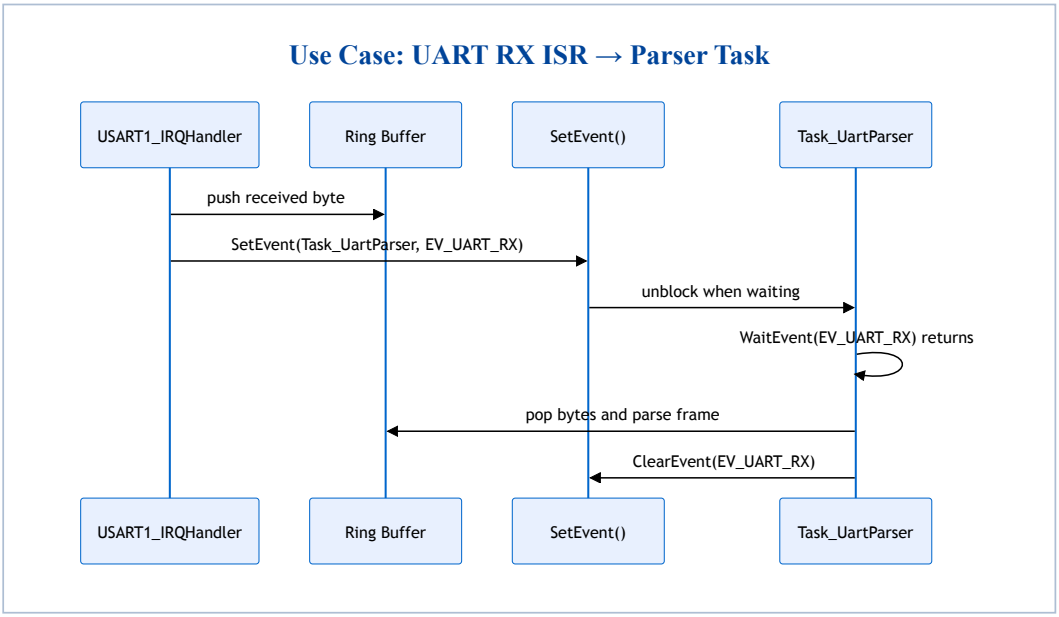
Khi Consumer đọc `GetEvent()`, mask kỳ vọng là `0x00000003`. Học viên cần sửa Consumer để xử lý từng bit:

```
if ((events & EVENT_RX_DONE) != 0u) {
    ClearEvent(EVENT_RX_DONE);
}
if ((events & EVENT_TIMEOUT) != 0u) {
    ClearEvent(EVENT_TIMEOUT);
}
```

10. Ứng dụng thực tế trong hệ automotive

10.1 UART RX: ISR đánh thức parser task

Ứng dụng kinh điển nhất là UART RX. ISR không nên parse frame dài. ISR chỉ ghi byte vào ring buffer và set event để task xử lý sau.



Thiết kế đúng là event chỉ báo "có dữ liệu", còn dữ liệu nằm trong ring buffer. Nếu ISR nhận 10 byte liên tiếp, event bit vẫn chỉ là 1, nhưng ring buffer giữ đủ 10 byte.

10.2 CAN RX: CanIf báo event cho application task

Trong gateway automotive, CAN driver hoặc CanIf nhận PDU rồi đánh thức task xử lý application. Event giúp tách phần thời gian gấp trong ISR khỏi phần xử lý nghiệp vụ.

Thành phần	Việc nên làm	Việc không nên làm
CAN RX ISR	Đọc mailbox, copy PDU vào buffer, gọi `SetEvent(Task_Com, EV_CAN_RX)`.	Không chạy toàn bộ diagnostic/application logic trong ISR.
Task_Com	`WaitEvent(EV_CAN_RX | EV_COM_TIMEOUT)`, xử lý PDU, clear event.	Không polling mailbox liên tục nếu không có dữ liệu.
Task_Dcm	Được báo event khi có diagnostic request hoàn chỉnh.	Không phụ thuộc trực tiếp vào interrupt timing.

10.3 Timeout và watchdog supervision

Alarm hoặc ScheduleTable có thể gọi `SetEvent(Task_X, EV_TIMEOUT)`. Task chờ cả data và timeout trong một API:

```
WaitEvent(EV_DATA_READY | EV_TIMEOUT);
GetEvent(TASK_X, &events);

if ((events & EV_DATA_READY) != 0u) {
    ClearEvent(EV_DATA_READY);
    ProcessData();
}

if ((events & EV_TIMEOUT) != 0u) {
    ClearEvent(EV_TIMEOUT);
    HandleTimeout();
}
```

Đây là pattern rất quan trọng: một task có thể chờ nhiều nguyên nhân đánh thức, rồi phân nhánh xử lý theo bit.

10.4 Diagnostic session

Trong diagnostic, event có thể báo các trạng thái như request mới, timeout P2/P2*, security seed ready, hoặc response transmit done. Nhờ event, DCM task không cần chạy liên tục ở mỗi tick.

Event	Nguồn phát	Task nhận	Hành động
`EV_DIAG_REQ`	CanTp indication	Task_Dcm	Parse UDS request.
`EV_DIAG_TIMEOUT`	Alarm/counter	Task_Dcm	Abort pending request hoặc gửi negative response.
`EV_TX_CONFIRM`	CanIf TxConfirmation	Task_Dcm	Chuyển state machine sang idle hoặc next block.

11. Debug chuyên sâu bằng GDB

11.1 Breakpoint nên đặt

Breakpoint	Mục tiêu quan sát	Lệnh gợi ý
`Task_EventConsumer`	Task bắt đầu, chuẩn bị gọi `WaitEvent`.	`hbreak Task_EventConsumer`
`WaitEvent`	Xem `g_current`, `event_set`, `wait_mask` trước khi block.	`hbreak WaitEvent`
`SetEvent`	Xem target task được unblock.	`hbreak SetEvent`
`os_schedule`	Xem scheduler chọn `g_next`.	`hbreak os_schedule`
`PendSV_Handler`	Xem context switch thật.	`hbreak PendSV_Handler`

11.2 Lệnh GDB nên dùng khi dạy

```
p/x g_current
p/x g_next
p ((TCB_t*)g_current)->id
p ((TCB_t*)g_current)->state
p/x ((TCB_t*)g_current)->event_set
p/x ((TCB_t*)g_current)->wait_mask
info registers psp msp lr pc
bt
```

Khi dừng ở `SetEvent`, không chỉ nhìn biến local. Hãy in TCB của Consumer để thấy event bit thay đổi:

```
p/x tcb[TASK_EVENT_CONSUMER].event_set
p/x tcb[TASK_EVENT_CONSUMER].wait_mask
p tcb[TASK_EVENT_CONSUMER].state
```

> Nếu `tcb` là static trong `os_kernel.c`, GDB vẫn có thể thấy symbol khi build có debug info. Nếu không thấy, có thể dùng breakpoint trong cùng file hoặc tạo helper debug public sau này.

11.3 Checklist debug khi flow sai

Triệu chứng	Khả năng lỗi	Cách kiểm tra
Consumer không chạy lại sau `SetEvent`.	Không enqueue khi unblock, hoặc priority không đủ để preempt.	Dừng ở `SetEvent`, kiểm tra `state`, ready queue, `g_next`.
`WaitEvent` không block.	Event bit đã pending do chưa clear.	In `event_set` trước `WaitEvent`.
Task chạy lại từ đầu thay vì sau `WaitEvent`.	Stack/context bị init lại sai.	Kiểm tra `context_needs_init` và PendSV save/restore.
Log thứ tự bị đảo.	Priority hoặc scheduler mode thay đổi.	Kiểm tra `OS_SCHED_MODE_DEFAULT` và `base_priority`.
HardFault sau unblock.	Stack frame hoặc PSP hỏng.	Breakpoint `PendSV_Handler`, kiểm tra PSP và TCB `sp`.

12. Hướng mở rộng lên OSEK/AUTOSAR

Bài này cố ý là primitive nền, chưa phải OSEK chuẩn đầy đủ. Khi học OSEK sau, các khái niệm sẽ được siết chặt thêm.

Bản nền hiện tại	OSEK/AUTOSAR sau này	Ý nghĩa mở rộng
API trả `void`.	API trả `StatusType`.	Cần báo `E_OS_ID`, `E_OS_ACCESS`, `E_OS_CALLEVEL`, v.v.
Mọi task có thể dùng event field.	Phân biệt basic task và extended task.	Chỉ extended task mới wait event.
Không kiểm tra calling context đầy đủ.	Quy định task context, ISR context rõ ràng.	`WaitEvent` không được gọi từ ISR.
Không có service ID/error hook.	Có ErrorHook, service ID, parameter capture.	Debug lỗi API chuẩn hơn.
ScheduleTable chỉ ActivateTask.	ScheduleTable có thể SetEvent.	Time-triggered event wakeup cho task.

12.1 Khi nào nên refactor tiếp?

- Khi cần negative test cho task ID sai, mask sai, caller sai context.
- Khi cần mô phỏng OSEK extended task đúng nghĩa.
- Khi ScheduleTable cần action `SetEvent` thay vì chỉ `ActivateTask`.
- Khi ISR category 2 cần gọi service OS có kiểm soát.
- Khi cần ErrorHook để dạy lỗi runtime.

13. Câu hỏi kiểm tra hiểu bản chất

1. Vì sao `WaitEvent()` phải check `event_set` trước khi chuyển task sang `WAITING`?
2. Vì sao `SetEvent()` không luôn gọi scheduler, đặc biệt khi đang trong ISR?
3. Nếu Producer set event cho Consumer khi Consumer đang READY, chuyện gì xảy ra?
4. Nếu Consumer quên `ClearEvent()`, lần `WaitEvent()` sau có block không?
5. Event bit có dùng để đếm 5 CAN frame nhận liên tiếp được không? Nếu không, cần thêm cấu trúc gì?
6. Vì sao task được unblock không đồng nghĩa chắc chắn chạy ngay?
7. Sau `SetEvent()`, context switch thật xảy ra trong function C hay trong `PendSV_Handler`?
8. Vì sao `GetEvent()` không được tự clear bit?
9. Race "lost wakeup" là gì trong cặp `WaitEvent` / `SetEvent`?
10. Nếu muốn ScheduleTable đánh thức task bằng event, cần thêm gì vào cấu trúc expiry point?

14. Điểm dễ nhầm và kết luận

- `SetEvent()` không trực tiếp gọi function của task đích. Nó chỉ set bit và có thể đưa task về ready queue.
- `WaitEvent()` không clear event. Nếu event đã pending, hàm return ngay.
- `GetEvent()` chỉ đọc, không thay đổi state.
- `ClearEvent()` nên được gọi sau khi task đã xử lý bit tương ứng.
- Event bitmask không đếm số lần xảy ra. Set cùng một bit nhiều lần vẫn chỉ là bit 1.
- Task đang `WAITING` không được scheduler chọn cho đến khi có event khớp.

Kết luận kỹ thuật: Sau bài này, kernel không còn chỉ là scheduler chạy task theo ready queue. Nó đã có primitive blocking đầu tiên: task có thể tự rời CPU khi chưa có việc và quay lại khi được báo event. Đây là nền quan trọng trước khi học OSEK event service đầy đủ.

Os_Test Event API Execution Flow - HALA Academy